KỸ THUẬT LẬP TRÌNH (INFO3008)

Tư duy Thiết kế trong Lập trình

Từ "Robot ngốc nghếch" đến Bộ ngôn ngữ hình ảnh chuẩn ISO/UML

Gemini Notebook

Trải nghiệm: Trò chơi "Robot ngốc nghếch"

Luật chơi:

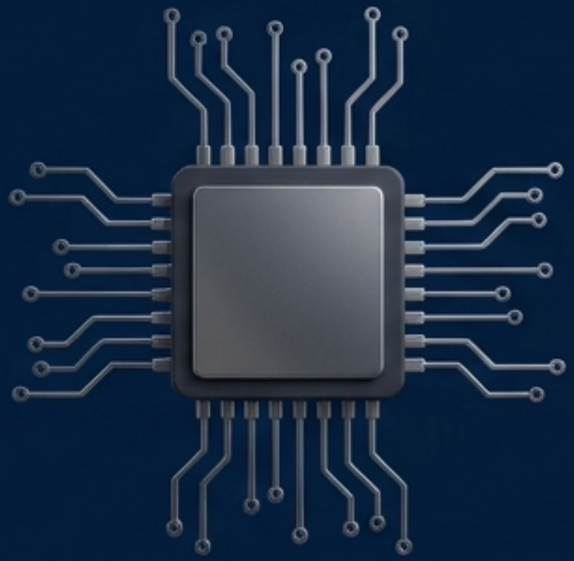
Sinh viên sử dụng ngôn ngữ tự nhiên để điều khiển Giảng viên (đóng vai Robot) đi đến lấy chiếc ly.

Cơ chế:

Robot không có “trí tuệ nền”. Robot thực hiện CHÍNH XÁC nghĩa đen của từng từ được ra lệnh. Không hơn, không kém.

Ai sẽ là lập trình viên đầu tiên ra lệnh?

Tại sao Robot liên tục thất bại?



Tư duy Con người

- Dựa trên ngữ cảnh (Context-dependent)
- Hiểu các ý ngầm (Implicit)
- Dễ dãi với sự mơ hồ (Tolerates ambiguity)

Cơ chế Máy tính

- Chỉ dựa trên dữ liệu (Data-driven)
- Cần chỉ thị rõ ràng tuyệt đối (Explicit)
- Thực thi cứng nhắc, từng bước (Rigid, step-by-step execution)

Máy tính không hiểu "ý định" của bạn. Nó chỉ thi hành "mệnh lệnh" của bạn.

Gemini Notebook

Cầu nối giao tiếp: Thuật toán (Algorithm)

Các bước hành động rõ ràng

Thứ tự thực thi chính xác

Thuật toán

Thuật toán = [Các bước hành động rõ ràng] + [Thứ tự thực thi chính xác]

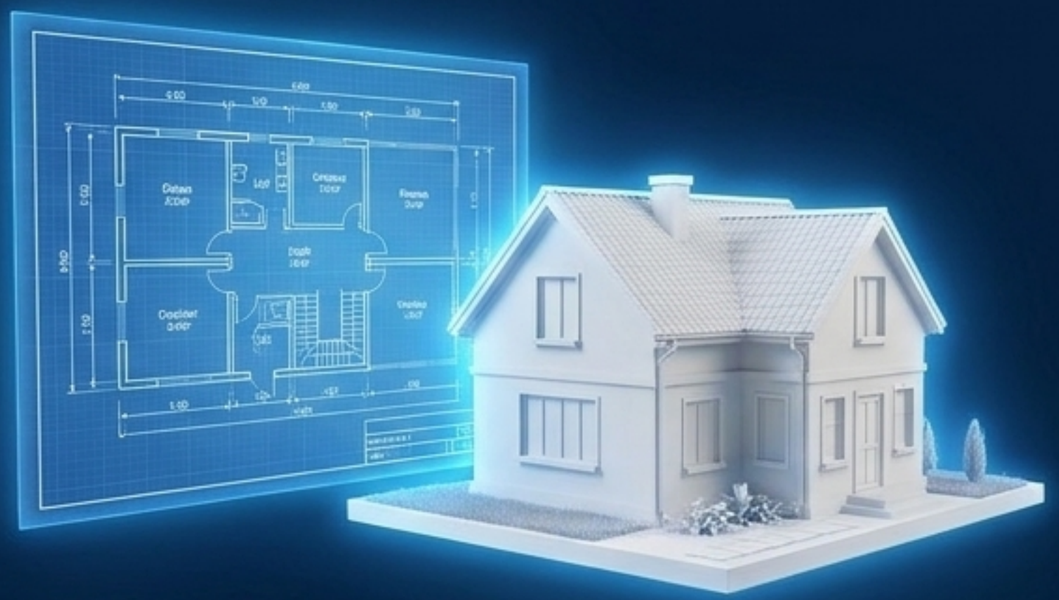
Không có thuật toán, mọi ngôn ngữ lập trình (C#, Python, Java) đều vô nghĩa.
Nó là phương pháp giải quyết bài toán trước khi bạn viết bất kỳ dòng code nào.

Gemini Notebook

Bạn không thể xây nhà nếu thiếu Bản vẽ kỹ thuật

TRƯỚC

SAU



Việc lập trình cũng vậy. Sơ đồ khối (Flowchart) chính là bản thiết kế của mọi kỹ sư phần mềm. Sơ đồ khối trực quan hóa luồng thông tin và các bước xử lý thiật thuật toán bằng một "bộ từ vựng" hình ảnh chuẩn quốc tế.

Gemini Notebook

01. Khởi đầu & Kết thúc (Terminal)

Chức năng:

Xác định điểm Bắt đầu (Entry point) và điểm Kết thúc (Exit point) của một thuật toán.

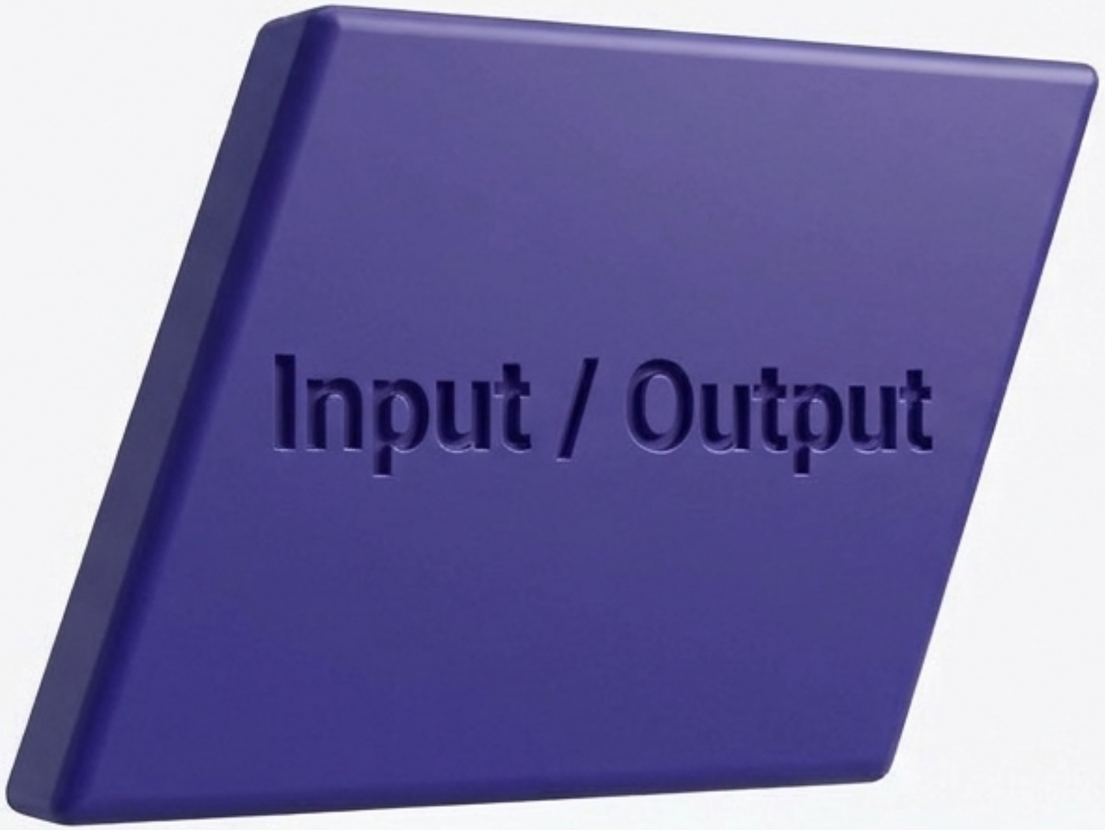
Quy tắc:

Mỗi sơ đồ khối chỉ có duy nhất MỘT điểm Start, nhưng có thể có nhiều điểm End.

Ẩn dụ:

Đóng vai trò như cánh cửa mở ra và đóng lại của một căn phòng.

Gemini Notebook



02. Nhập & Xuất dữ liệu (Input / Output)

Chức năng:

Sự giao tiếp giữa con người và máy tính. Nơi máy tính nhận "nguyên liệu" (Input) và trả về "thành phẩm" (Output).

Ví dụ:

- Input: Nhập mật khẩu, Tải ảnh lên.
- Output: In ra màn hình "Đăng nhập thành công".

Gemini Notebook

03. Xử lý & Tính toán (Action State / Process)

Chức năng:
Đại diện cho một hành động, phép gán giá trị, hoặc tính toán toán học. Luôn bắt đầu bằng một ĐỘNG TỪ.

Ví dụ:
- Tính Tổng = A + B
- Gán biến đếm = đếm + 1

Lưu ý:
Nơi “phép thuật” CPU thực sự hoạt động. Không có giao tiếp với người dùng ở đây.

Gemini Notebook

04. Quyết định (Decision)

Chức năng:
Kiểm tra một điều kiện logic. Dòng chảy thuật toán sẽ chia nhánh dựa trên kết quả Đúng hoặc Sai.

Quy tắc:
Luôn chứa một câu hỏi Có/Không. Có một mũi tên đi vào, nhưng luôn có HAI mũi tên đi ra.

Chúng ta sẽ khám phá sức mạnh của khối này trong bài "Cấu trúc Rẽ nhánh" tuần sau.

Gemini Notebook

05. Định hướng dòng chảy (Flowline / Transition)

Chức năng:

- Trục thời gian của thuật toán.
- Xác định chính xác bước nào phải diễn ra trước, bước nào diễn ra sau.

Quy tắc:

- Mũi tên không bao giờ được phép cắt chéo nhau gây nhầm lẫn.
- Luôn đi từ trên xuống dưới, hoặc từ trái sang phải.

Gemini Notebook

Ma trận Ngôn ngữ Sơ đồ khối (ISO/UML Standard)

Ký hiệu	Tên gọi	Chức năng (Động từ)
	Terminal	Bắt đầu / Kết thúc chương trình
	Input / Output	Nhập / Xuất dữ liệu
	Process	Tính toán / Gán biến
	Decision	Đánh giá điều kiện (Đúng/Sai)
	Flowline	Định hướng luồng thực thi

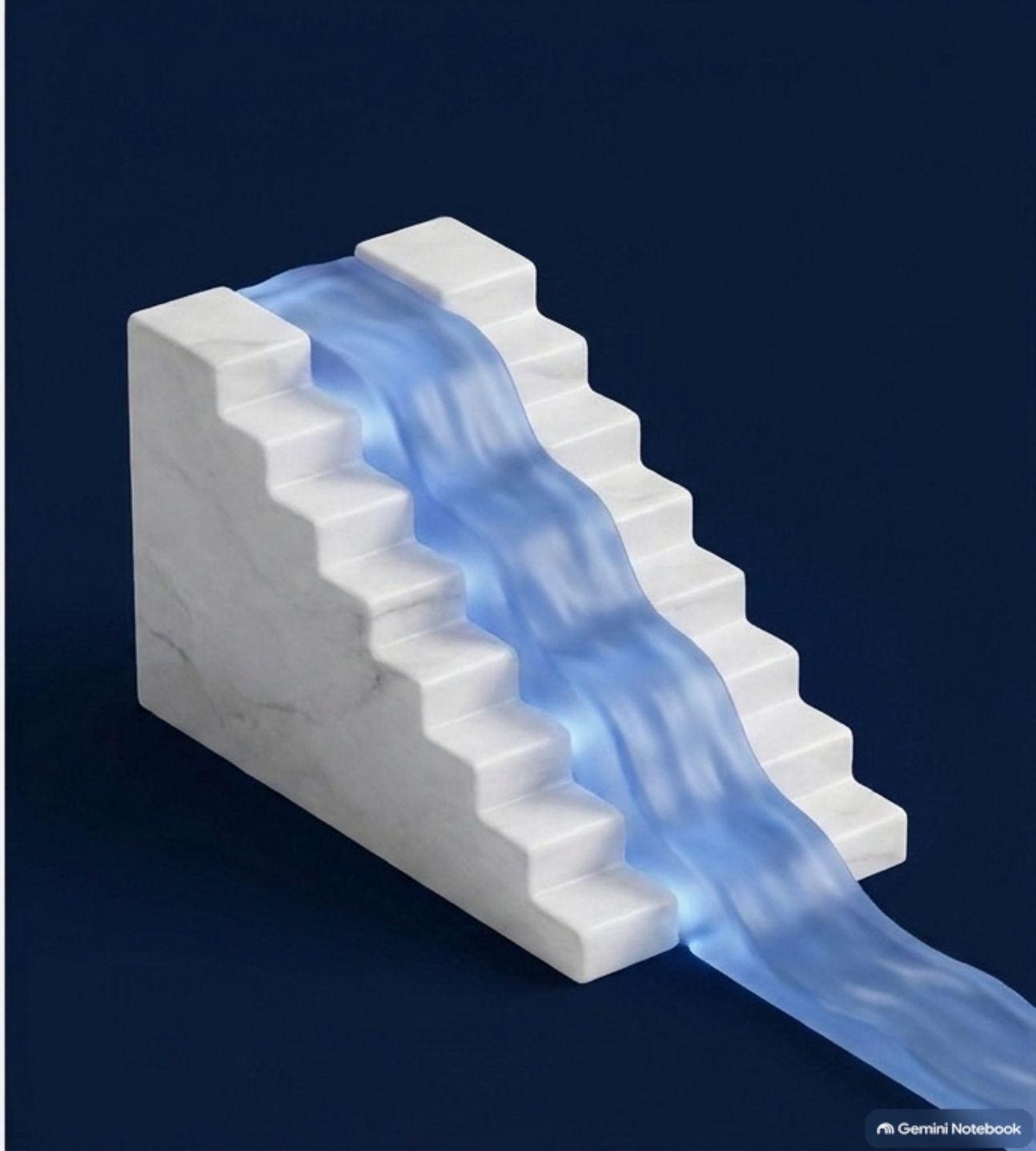
Gemini Notebook

Cấu trúc nền tảng: Tuần tự (Sequential Structure)

Máy tính mặc định thi hành các lệnh nối tiếp nhau, từ trên xuống dưới.

Nguyên tắc 3 KHÔNG:

1. Không đảo ngược thứ tự.
2. Không nhảy cóc (bỏ qua) bất kỳ khối nào.
3. Không lặp lại lệnh đã chạy.

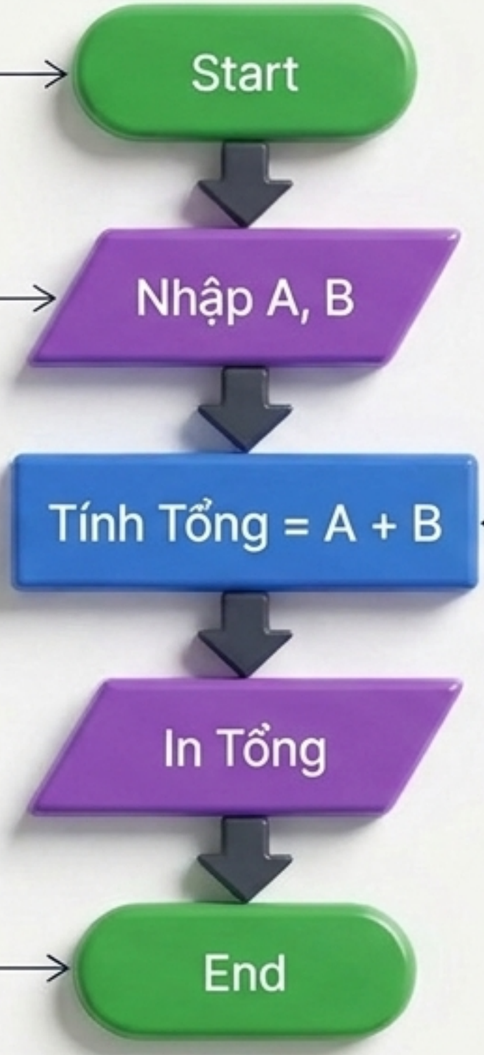


Gemini Notebook

Giải phẫu Cấu trúc Tuần tự

Điểm bắt buộc khởi tạo hệ thống.

Chờ thao tác từ người dùng.



CPU hoạt động ngầm.

Đóng quy trình giải phóng tài nguyên.

Gemini Notebook

Thực hành: Lắp ráp Bản thiết kế đầu tiên

Tình huống: Viết chương trình tính tuổi của người dùng.

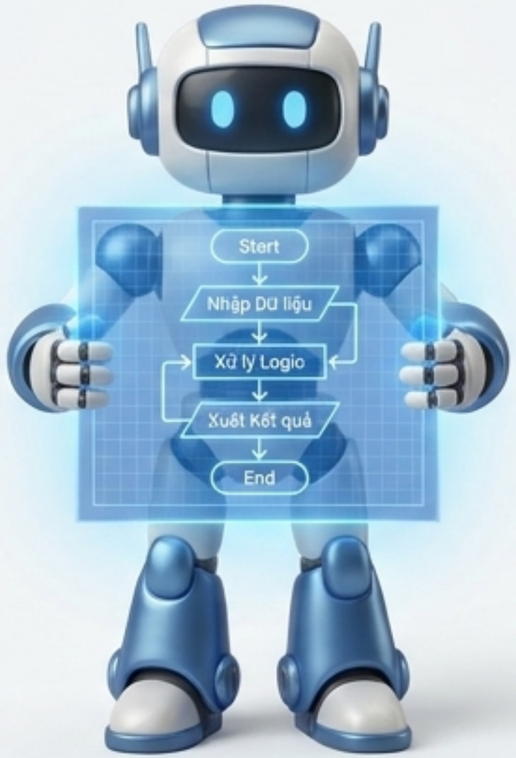
Đề bài: Hãy sử dụng 5 ký hiệu ISO/UML vừa học để vẽ Sơ đồ khối (Flowchart) cho thuật toán sau:

1. Yêu cầu người dùng nhập Năm Sinh.
2. Tính Tuổi = Năm Hiện Tại (2026) - Năm Sinh.
3. Xuất kết quả Tuổi ra màn hình.

Thời gian: 5 Phút.

Gemini Notebook

Lập trình là Tư duy Thiết kế (Design Thinking)



"Việc viết code (C#, Python) chỉ là quá trình dịch thuật. Tư duy xây dựng thuật toán bằng sơ đồ khối mới là cốt lõi của một Kỹ sư phần mềm thực thụ."

Hôm nay chúng ta học cách chảy xuôi dòng (Tuần tự). Tuần sau, chúng ta sẽ cấp cho hệ thống "quyền năng lựa chọn" với Cấu trúc Rẽ nhánh! (If-Else).

Gemini Notebook

Thử thách Logic: Rẽ nhánh, Vòng lặp & Hệ thống ATM

Tiết 2 - Kỹ thuật lập trình C# (INFO3008)

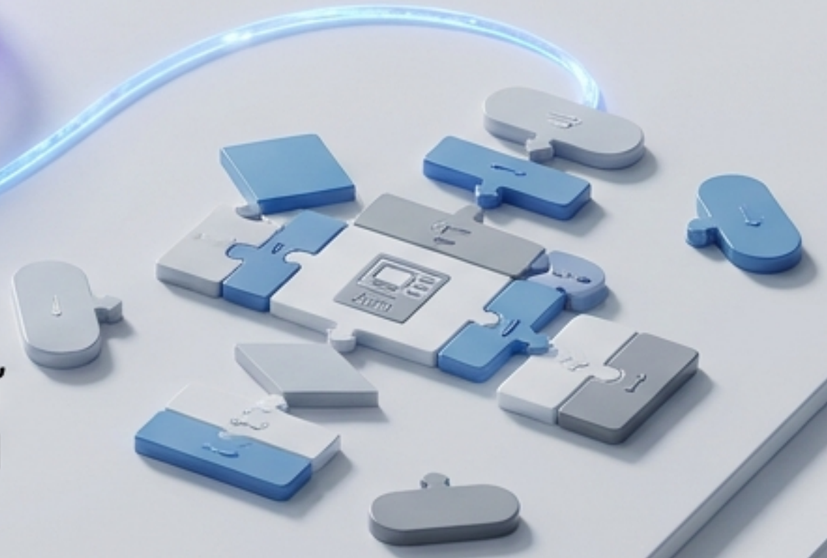
Gemini Notebook

Hành trình Tư duy Hôm nay

Trạm 1: Ngã rẽ Quyết định
Cấu trúc Rẽ nhánh (If-Else / Switch)

Trạm 2: Vòng lặp Thời gian
Cấu trúc Lặp (For / While)

Trạm 1: Vòng lặp Thời gian
Cấu trúc Lặp (For / While)



Trạm 3: Thử thách Kỹ sư
Xây dựng Logic Máy ATM

Gemini Notebook

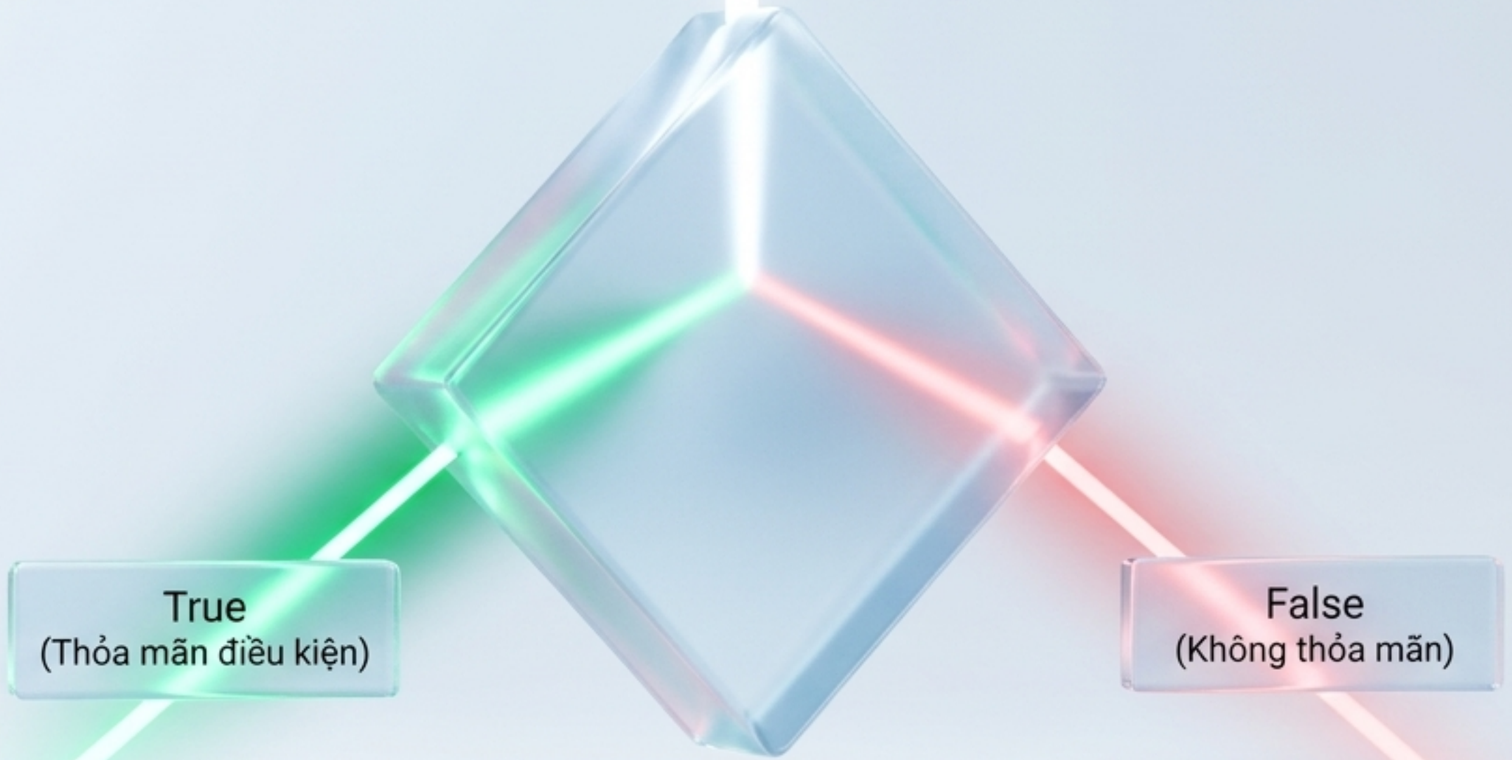
Chương trình không thể chỉ đi một đường thẳng

Khái niệm Rẽ nhánh dựa trên Điều kiện (Condition)

Bản chất: Điều kiện là một biểu thức luôn trả về giá trị Boolean (True hoặc False).

Toán tử quan hệ:
>, <, >=, <=, ==, !=

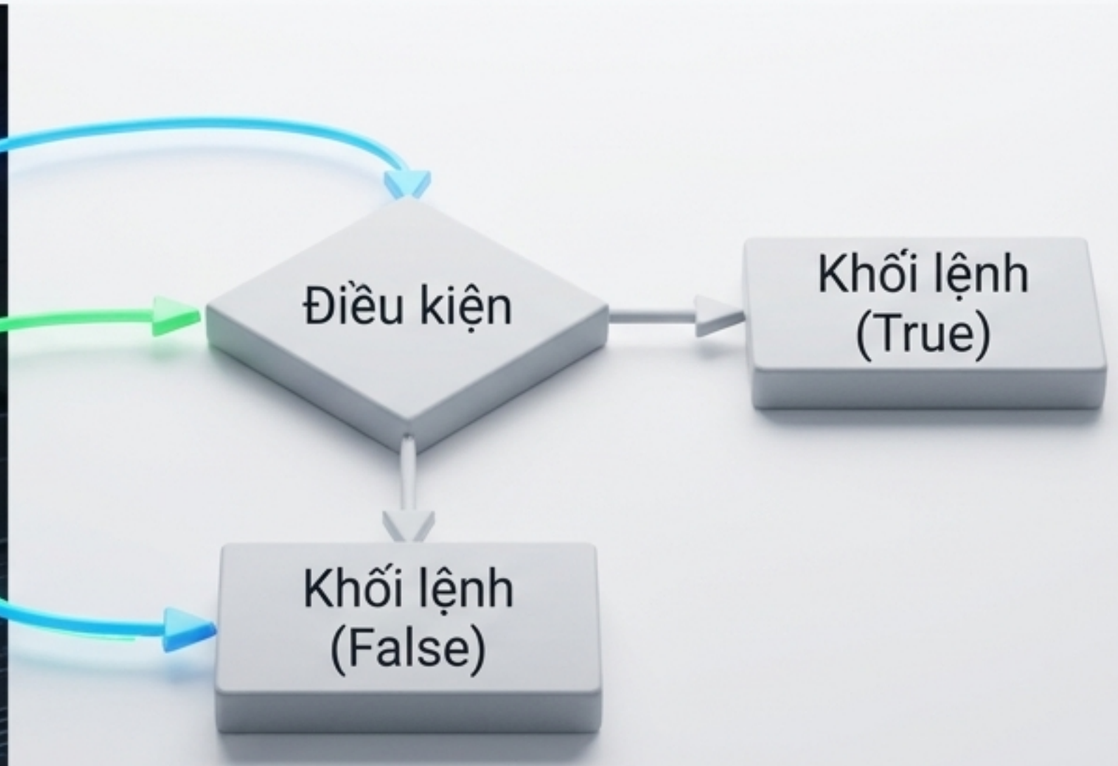
Hành động: Máy tính sẽ chọn ngã rẽ dựa trên kết quả của điều kiện này.



Gemini Notebook

Giải phẫu Cấu trúc If-Else

Kết nối giữa Mã nguồn C# và Sơ đồ khối

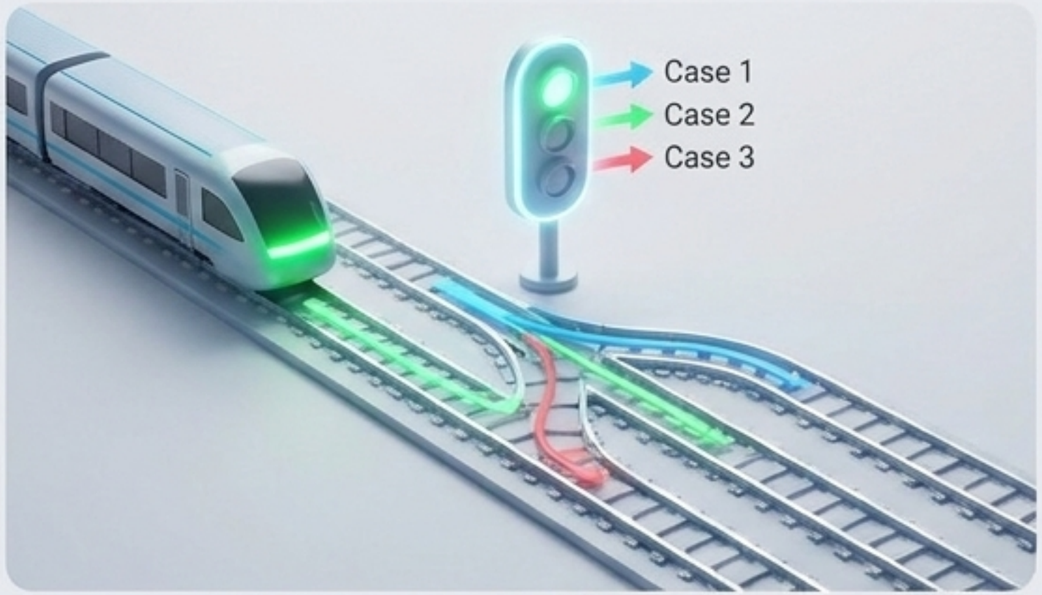


Lỗi Dangling-else: Trình biên dịch C# sẽ tự động ghép **else** với **if** gần nhất. Luôn sử dụng **ngoặc nhọn { }** để nhóm khối lệnh (Block) một cách an toàn!

Gemini Notebook

Đa lựa chọn: Switch-Case hay Nested If?

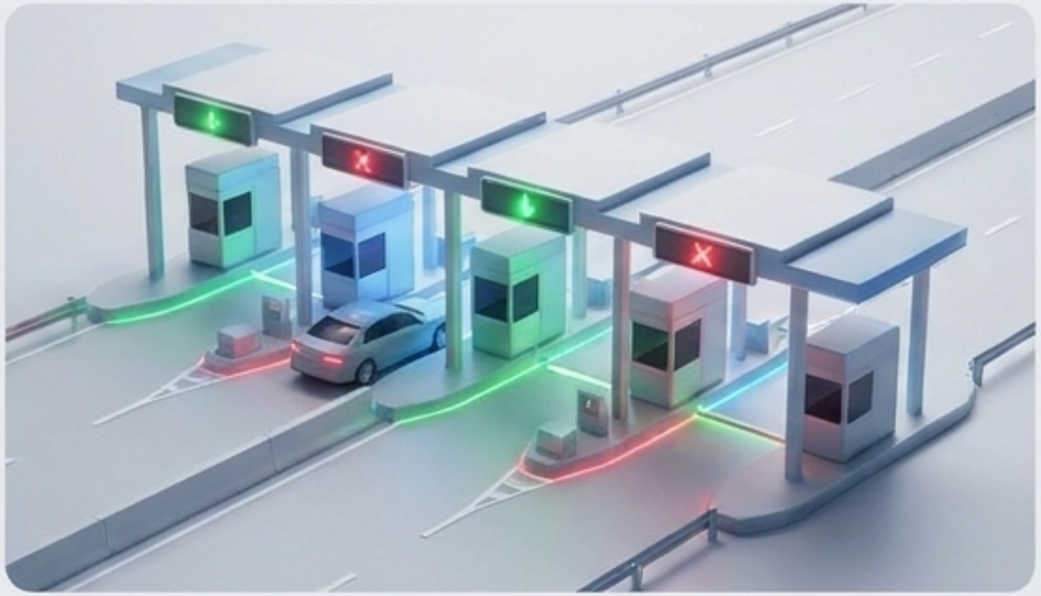
Cấu trúc **Switch**



Đặc điểm: Kiểm tra một biểu thức với các giá trị cụ thể (Constant values).

Quy tắc C#: Không có Fall-Through - Bắt buộc phải có lệnh break ở cuối mỗi case.

Cấu trúc **Nested If**



Đặc điểm: Kiểm tra các khoảng giá trị (Ranges) hoặc các điều kiện logic phức tạp (sử dụng &&, ||).

Gemini Notebook

Phá vỡ sự giới hạn bằng Vòng lặp

Cỗ máy tự động hóa việc lặp lại (Iteration)

Vòng lặp giúp chúng ta không phải viết cùng một dòng code 1000 lần.

1. Lặp biết trước số lần (Counter-controlled): Định tuyến rõ ràng, đếm số vòng (Definite iteration).

2. Lặp không biết trước (Sentinel-controlled): Lặp cho đến khi gặp một tín hiệu dừng (Indefinite iteration).

Gemini Notebook

Ma trận Lựa chọn Vòng lặp

For	**Kiểm tra:** Trước khi lặp	**Số lần tối thiểu:** 0	**Khi nào dùng:** Khi biết chính xác số lần cần lặp
While	**Kiểm tra:** Trước khi lặp	**Số lần tối thiểu:** 0	**Khi nào dùng:** Lặp dựa trên điều kiện, có thể không chạy lần nào
Do...While	**Kiểm tra:** Sau khi lặp	**Số lần tối thiểu:** 1 (Luôn chạy ít nhất 1 lần)	**Khi nào dùng:** Khi cần thực thi hành động trước, kiểm tra sau (VD: Nhập menu)

Gemini Notebook

Giải phẫu Luồng thực thi lệnh For

for (int i = 0; i < 10; i++)

1. Khởi tạo (Init):
int i = 0
(Chạy đúng 1 lần duy nhất)

2. Điều kiện (Condition):
i < 10
(Trạm kiểm soát trước khi vào vòng lặp)

3. Cập nhật (Increment): i++
(Thực hiện sau khi chạy xong khối lệnh)

Gemini Notebook

Điều khiển Luồng: Trượt vòng hay Bỏ cuộc?

Sự khác biệt cốt lõi giữa Continue và Break

Break

Thoát hoàn toàn khỏi vòng lặp.

Continue

Bỏ qua phần còn lại của bước hiện tại, nhảy ngay sang vòng lặp tiếp theo.

Gemini Notebook

Cỗ máy Hoàn chỉnh: Cấu trúc lồng nhau (Nesting)

Tư duy Hệ thống: Một quyết định (If) hoàn toàn có thể được thực hiện lặp đi lặp lại nhiều lần bên trong một chu trình (While). Đây chính là cốt lõi của mọi ứng dụng thực tế.

Gemini Notebook

Thử thách Nhóm: Trở thành Kỹ sư ATM

Vận dụng Rẽ nhánh & Vòng lặp vào Bài toán thực tế

Khách hàng: Ngân hàng Đông Á.

Nhiệm vụ: Xây dựng sơ đồ khối (Flowchart) cho hệ thống logic cốt lõi của máy ATM mới.

Mục tiêu: Áp dụng kết hợp cả 2 công cụ vừa học (Vòng lặp + Rẽ nhánh).

Gemini Notebook

Phân tích Yêu cầu Bài toán ATM

Xác thực (Authentication)

Nhập mã PIN. Nếu sai, yêu cầu nhập lại.

Cảnh báo: Khóa thẻ nếu sai quá 3 lần!

(Vòng lặp While hoặc Do...While)

Điều hướng (Navigation)

Hiển thị Menu:
1. Rút tiền | 2. Xem số dư.

(Cấu trúc Switch-Case)

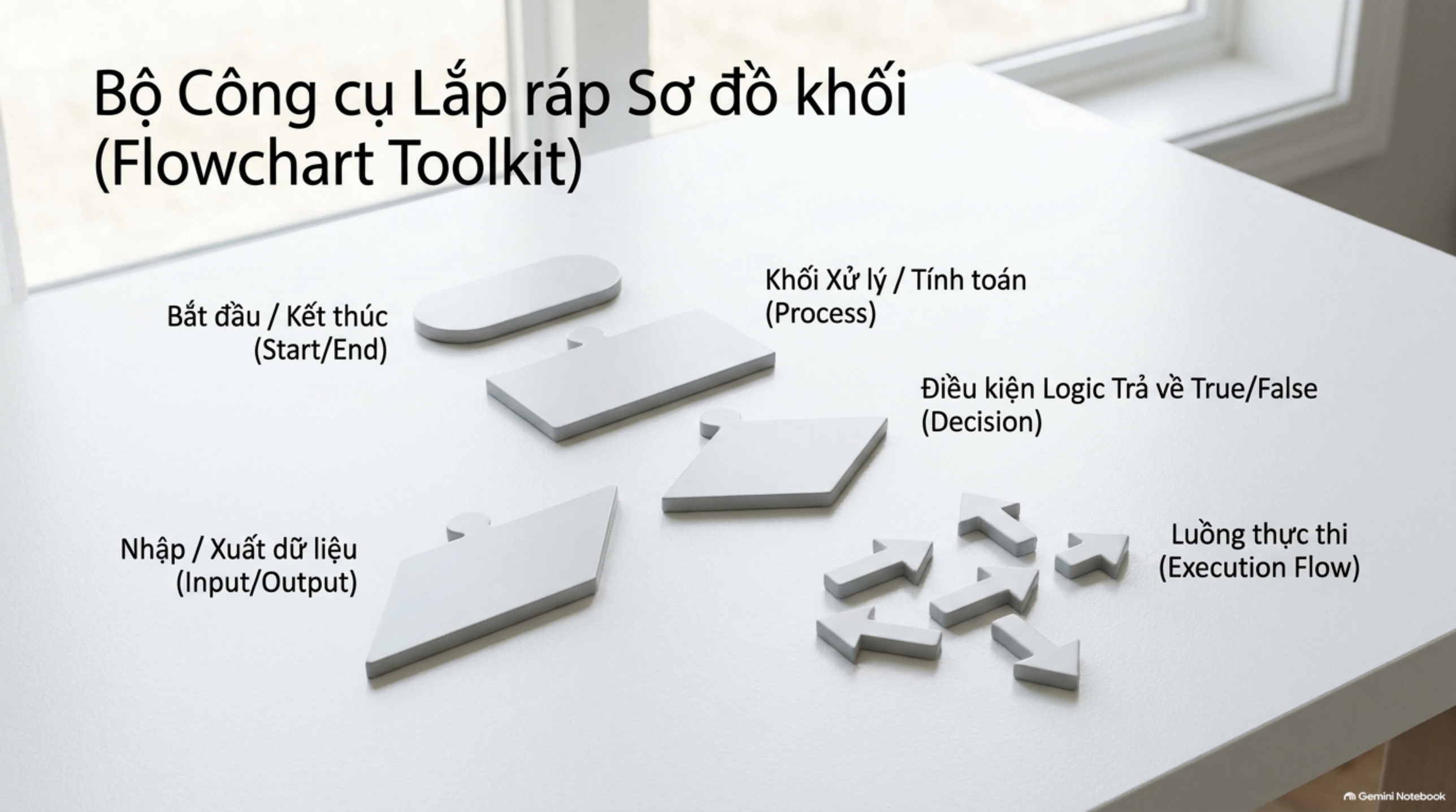
Giao dịch (Transaction)

Kiểm tra: Số dư có lớn hơn hoặc bằng số tiền cần rút?

(Cấu trúc If-Else)

Gemini Notebook

Bộ Công cụ Lắp ráp Sơ đồ khối (Flowchart Toolkit)



Gemini Notebook

Thời gian Lắp ráp! (Action Time)

1. **Lập team:** Chia nhóm 4-5 sinh viên.
2. **Công cụ:** Sử dụng bộ thẻ giấy được phát (hoặc công cụ vẽ online).
3. **Nhiệm vụ:** Ghép nối các khối để tạo thành luồng logic hoàn chỉnh cho máy ATM từ lúc đút thẻ đến lúc trả thẻ.
4. **Chấm điểm:** Chú ý logic vòng lặp nhập PIN và nhánh kiểm tra số dư.

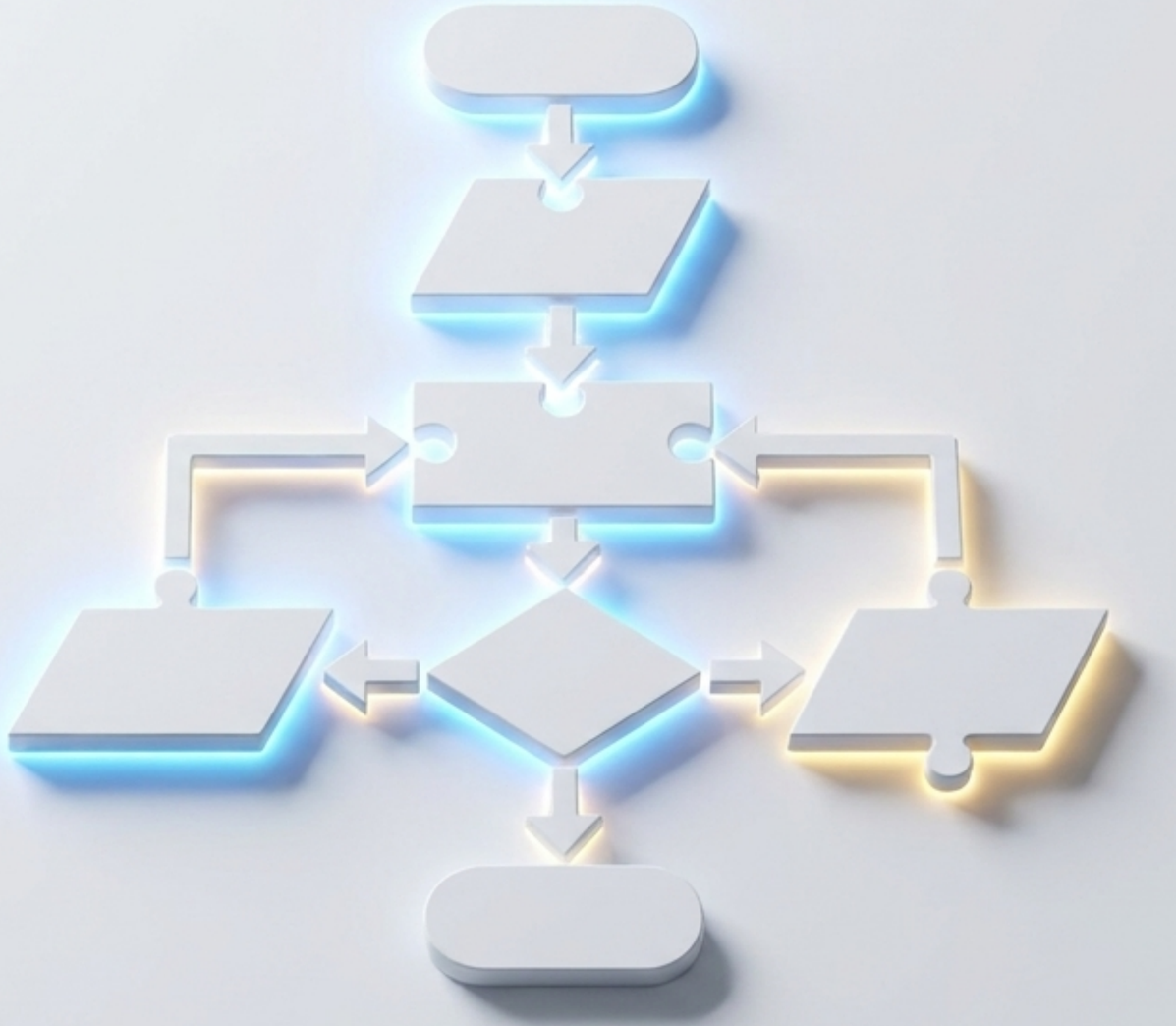
Gemini Notebook

Mảnh ghép Hoàn chỉnh: Từ Tư duy đến Mã nguồn

Bạn đã làm chủ: Khả năng bẻ gãy dòng thực thi tuần tự bằng **Rẽ nhánh (If/Switch)** và bẻ cong thời gian bằng **Vòng lặp (For/While)**.

Nhiệm vụ về nhà: Chuyển đổi sơ đồ khối ATM hoàn hảo này thành những dòng **code C#** thực tế (Sử dụng Visual Studio).

Mã nguồn là công cụ.
Tư duy logic mới là cốt lõi.



Gemini Notebook

MINI-HACKATHON: Flowchart Master & C# Code Connection

Biến Ý Tưởng Thành Hiện Thực

Tiết 3 | 45 Phút

Mục tiêu: Chuyển hóa tư duy logic (CLO3) thành mã nguồn thực tế (CLO2).

Gemini Notebook

Nhiệm Vụ Và Luật Chơi (45 Phút)

Làm việc theo nhóm. Mục tiêu là giải quyết 3 bài toán thực tế bằng cách tuân thủ nghiêm ngặt quy trình rèn giữa logic trước khi gõ code.

Giai đoạn 1: Bản Vẽ (15 Phút)
Thiết kế sơ đồ khối (Flowchart) trên giấy/bảng.

Giai đoạn 2: Lò Luyện (10 Phút)
Đổi bài và phản biện chéo trong vai trò Trình biên dịch (Compiler).

Giai đoạn 3: Hiện Thực Hóa (20 Phút)
Chuyển giao sang mã giả và viết code C# trên Visual Studio.

Gemini Notebook

Ma Trận Thử Thách Logic

Mỗi bài toán kiểm tra một nhóm cấu trúc điều khiển nền tảng trong lập trình C#.

Bài toán 1: Đèn Giao Thông

Vấn đề: Nhập trạng thái đèn, in ra hành động tương ứng.

Trọng tâm Logic: Rẽ nhánh đa hướng.

Cấu trúc C#: Lệnh switch hoặc if-else if.

Bài toán 2: Tìm Số Lớn Nhất

Vấn đề: So sánh 3 số nguyên (a, b, c) để tìm ra số lớn nhất.

Trọng tâm Logic: Tuần tự & Logic so sánh.

Cấu trúc C#: Lệnh if lồng nhau & Toán tử &&.

Bài toán 3: Game Đoán Số

Vấn đề: Đoán số ngẫu nhiên (1-10) cho đến khi chính xác.

Trọng tâm Logic: Vòng lặp không xác định trước số lần.

Cấu trúc C#: Lệnh lặp while hoặc do-while.

Gemini Notebook

Bộ Công Cụ Dựng Sơ Đồ Khối

Bắt đầu / Kết thúc (Start/End)
Định tuyến điểm đầu và cuối của chương trình.

Nhập / Xuất (Input/Output)
Giao tiếp với người dùng (đọc dữ liệu, in kết quả).

Xử lý (Process)
Các phép gán, tính toán toán học.

Điều kiện (Decision)
Rẽ nhánh logic dựa trên kết quả True/False.

Gemini Notebook

Thử Thách 1: Hệ Thống Đèn Giao Thông

Yêu cầu bài toán:

Chương trình yêu cầu người dùng nhập vào màu của đèn giao thông hiện tại (**Đỏ, Vàng, Xanh**).

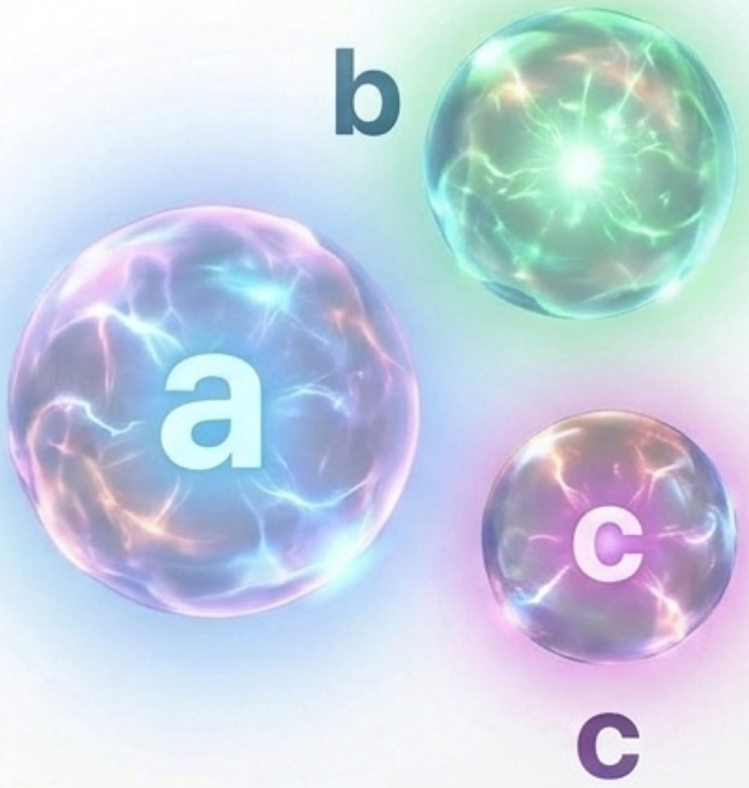
Dựa vào đầu vào, hệ thống phải in ra chỉ thị hành động tương ứng:

- **Đỏ**: Dừng lại
- **Vàng**: Đi chậm
- **Xanh**: Đi tiếp
- **Khác**: Trạng thái không hợp lệ

Gợi ý Logic: Từ một điểm nối, luồng xử lý của bạn sẽ chia thành bao nhiêu nhánh?

Gemini Notebook

Thử Thách 2: Truy Tìm Số Lớn Nhất



Yêu cầu bài toán:

Viết chương trình nhận vào 3 số nguyên ngẫu nhiên **a**, **b**, và **c**.

Xác định và in ra màn hình giá trị lớn nhất trong 3 số đó.

Gợi ý Logic & Bẫy (Edge Cases):

- Làm sao để sơ đồ khối thể hiện việc so sánh **a** với cả **b** và **c**?
- Hãy cẩn thận: Điều gì xảy ra nếu **a == b**, hoặc cả 3 số bằng nhau? Sơ đồ khối của bạn có bị kẹt (dead-end) không?

Gemini Notebook

Thử Thách 3: Game Đoán Số Ngẫu Nhiên

Yêu cầu bài toán:

Hệ thống (máy tính) sẽ tạo ra một số bí mật từ 1 đến 10.

Người dùng liên tục nhập một số để đoán. Hệ thống sẽ lặp lại yêu cầu nhập cho đến khi người dùng đoán chính xác.

Gợi ý Logic:

- Khác với rẽ nhánh, luồng (flow) của vòng lặp phải quay ngược lại một bước trước đó. Khối điều kiện của bạn sẽ nằm ở đầu vòng lặp hay cuối vòng lặp?

Gemini Notebook

Đổi Bài! Hãy Đóng Vai Trình Biên Dịch

Con người

Sáng tạo, linh hoạt, hay tự suy diễn ý định.

Trình Biên Dịch (Compiler)

Lạnh lùng, chính xác, tuyệt đối tuân thủ.

Quy tắc của Compiler: Trình biên dịch **không có trí tưởng tượng**. Trình biên dịch không tự hiểu ý của bạn. Trình biên dịch chỉ di chuyển **tuần tự theo đúng những mũi tên được vẽ trên giấy**. Nếu **không có mũi tên**, nó sẽ **Crash**.

Gemini Notebook

Kiểm Tra Lỗi Logic (Human vs. Compiler)

Sử dụng bảng sau để dò lỗi trên sơ đồ khối của nhóm bạn:

Góc nhìn Con Người (Human)	Góc nhìn Trình Biên Dịch (Compiler)	Lỗi Cần Bắt (Bugs)
Đương nhiên biến này là số.	Biến này đã được khởi tạo giá trị chưa?	Lỗi cấp phát (Definite assignment)
Đường nào cũng ra kết quả.	Nếu Điều kiện là FALSE, nhánh này đi về đâu?	Kẹt luồng (Dead-end path)
Lặp đến khi đoán đúng thì thôi.	Có biến nào thay đổi để điều kiện thoát lặp thành TRUE không?	Vòng lặp vô hạn (Infinite Loop)

Gemini Notebook

Giải Mã: Từ Đồ Họa Đến Mã Nguồn

Mã giả (Pseudocode) là bước đệm an toàn giữa tư duy không gian và cú pháp C#.

1. Ý Tưởng (Flowchart)

2. Mã Giả (Pseudocode)

```
NẾU x lớn hơn 5 THÌ
    Thực hiện khối lệnh
```

3. Mã C# (Syntax)

```
if (x > 5)
{
    // Thực hiện khối lệnh
}
```

Gemini Notebook

Hộp Đồ Nghề Cú Pháp C# (Sẵn Sàng Code)

Lắp ráp các cấu trúc sau vào Visual Studio để giải quyết 3 bài toán:

Nhập / Xuất

```
Console.ReadLine();
Console.WriteLine();
```

Chuyển kiểu (Parsing)

```
int.Parse(...)
```

Rẽ nhánh & Lựa chọn

```
if (...) { }
} else { }

switch (biến) {
    case x: break;
}
```

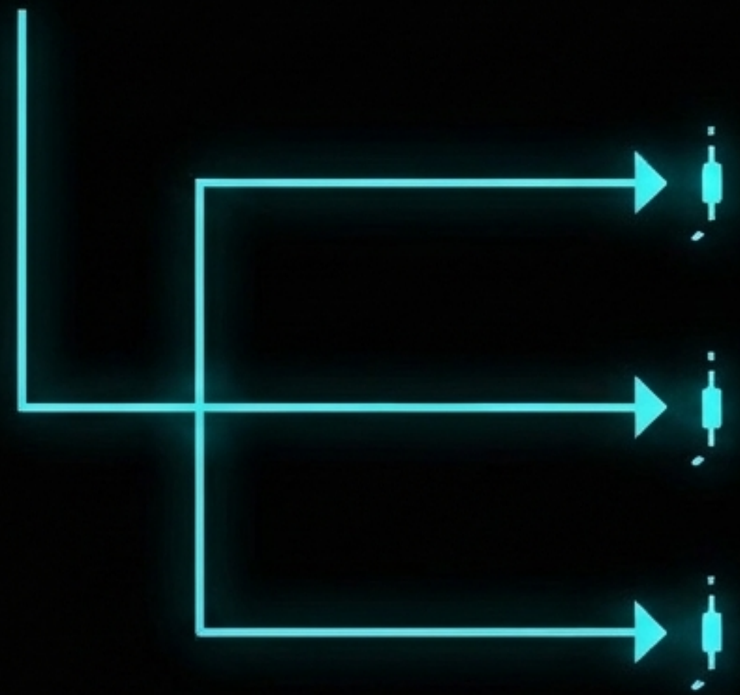
Vòng lặp

```
while (điều kiện) { }
do { }
} while (điều kiện);
```

Gemini Notebook

Giải Pháp 1: Lựa Chọn Đa Hướng (Switch)

Thay vì dùng nhiều lệnh if-else lồng nhau gây rối mắt, switch kiểm tra một giá trị rời rạc với độ đọc hiểu cao hơn.



```
string color = Console.ReadLine();
switch (color)
{
    case "Đỏ":
        Console.WriteLine("Dừng lại");
        break;
    case "Vàng":
        Console.WriteLine("Đi chậm");
        break;
    case "Xanh":
        Console.WriteLine("Đi tiếp");
        break;
    default:
        Console.WriteLine("Lỗi");
        break;
}
```

Gemini Notebook

Giải Pháp 2: Toán Tử Logic (&&)

Hai khối điều kiện lồng nhau trong sơ đồ khối có thể được gộp lại thành một biểu thức duy nhất bằng toán tử AND (&&).

```
int a = int.Parse(Console.ReadLine());
int b = int.Parse(Console.ReadLine());
int c = int.Parse(Console.ReadLine());

if (a >= b && a >= c) {
    Console.WriteLine($"Lớn nhất là: {a}");
} else if (b >= a && b >= c) {
    Console.WriteLine($"Lớn nhất là: {b}");
} else {
    Console.WriteLine($"Lớn nhất là: {c}");
}
```

Gemini Notebook

Giải Pháp 3: Vòng Lặp Post-Test (do-while)

Vì sao dùng do-while thay vì while? Trò chơi yêu cầu người dùng phải nhập số đoán ít nhất một lần trước khi kiểm tra điều kiện thắng.

```
Random rnd = new Random();
int secret = rnd.Next(1, 11);
int guess;

do
{
    Console.Write("Đoán số (1-10): ");
    guess = int.Parse(Console.ReadLine());
} while (guess != secret);

Console.WriteLine("Chính xác!");
```

Gemini Notebook

Tổng Kết: Quy Trình Kỹ Thuật Lập Trình

1. **Logic đi trước Cú pháp**: Vẽ sơ đồ khối để triệt tiêu lỗi tư duy trước khi chạm vào bàn phím.
2. **Thiết kế phòng thủ:** Trình biên dịch hiểu nghĩa đen. Hãy khởi tạo biến và kiểm soát điều kiện thoát vòng lặp.
3. **Lắp ghép nền tảng:** If/Else, Switch, và Vòng lặp chính là những viên gạch xây dựng nên mọi thuật toán phức tạp nhất.

Nhiệm vụ tuần tới: Ứng dụng các vòng lặp này để duyệt và xử lý dữ liệu với Mảng (Arrays)!

Gemini Notebook